Master's Program in International Cybersecurity and Cyberintelligence

2025/2026

Report for the Course on

Cyber-Physical Systems and IoT Security

Title of the reference paper:

Authentication of IoT Device and IoT Server Using Secure Vaults, IEEE TrustCom/BigDataSE 2018.

2169583 — Jan Clemens

2185102 — Valentin Andre Maier

2185202 — Max Olaf Opperman

Introduction

In an interconnected world, secure authentication is an integral challenge. While each individual *thing* in the internet of things (IoT) is only a small part contributing to a huge network, each device needs to be able to connect to a server in a reliable way. If a device is compromised, it can have real world consequences for consumers. Those consequences can be manifold: any device with a sensor like a microphone or a camera can be used for malicious purposes, a device that is also connected to a local wireless network can become the entry point for other threats, and some IoT devices also have cyber-physical capabilities like opening doors.

There are likely as many good examples on how an IoT device can be misused as there are bad authentication protocols. Say for instance, a company uses the same password to authenticate many (or all) of their IoT devices. If the password is retrieved from one of the devices, all the other devices can also potentially be accessed. It is somewhat safe to say that password-based authentication cannot compete with more sophisticated authentication schemes like the secure vault approach that is presented in the paper referenced.

The paper “Authentication of IoT Device and IoT Server Using Secure Vaults” (Sha and Venkatesan 2018) promotes an approach that mitigates most of the risks that come with common authentication mechanisms. The described protocol functions by continuously evolving two different vaults. A vault is a data structure containing several keys. Each communication between a device and the server and a device begins with a handshake that ensures the integrity of the vaults on both sides. At the end of communication, new vaults are generated using information exchanged between the devices.

As Sha and Venkatesan (2018) describe, this approach is secure to several different kinds of attacks. Namely, it is secure against Machine-In-The-Middle[1] (MITM) attacks, *Next Password Predictions*, some side channel attacks, and *Denial of Service (DoS)* attacks. As part of this project is to prove the security of the protocol using the Tamarin prover, its resilience against these mentioned attacks will be discussed later.

While the original paper describes the mechanics of the handshake in great detail, it is far from a complete implementation manual. For instance, the vault itself is described as somewhat as black box. Also, the entire protocol runs on the assumption that the network is stable and not subject to interference. These among other implementation details will be discussed right after explaining the objectives of this work.

Objectives

There are several different achievements in Sha and Venkatesan’s (2018) paper that could be recreated here. As not all of them make sense to recreate, we limited this work to providing a functional implementation of the authentication protocol and proving that it is as secure as the authors proclaim using the Tamarin prover. While we believe that the secure vault approach is highly efficient compared to other public key cryptosystems, there have been more efficient

[1] While the authors specifically mention a *man* in the middle, we believe our implementation is safe against any kind of attacker and therefore refer to a *machine* in the middle instead.

methods developed (Roy et al. 2024). Therefore, we did not see the value of measuring the time or energy efficiency of this authentication method.

System Setup

Software:

Python 3.14.3

PyCryptodome 3.23.0

Flask 3.1.2

Tamarin 1.10.0

The implementation of the secure vault is a proof-of-concept. The code attached is written in plain Python to be executable in most environments. While deploying the client code to a microcontroller (like ESP32) or a single-board-computer (like a Raspberry) is possible, it is not required as the client, and server code can simply be run in two different terminal windows. The current approach requires user input to have a more interactive demonstration.

Regarding the structure of the code, we have opted for a library and application code approach. All the functions that are specific to the secure vault authentication are defined in the **secure_vault** package. This package contains **server, client, vault,** and **utility** functions that can then be used by applications of this specific authentication protocol.

The server

The server application found in the **server** subdirectory waits for devices to connect to it. A connection request must follow a specific format, namely the structure of **M1**. Every request that the client sends includes a Session ID. Before responding to a request, the server checks if the referenced session is already correctly initialized. If not, it will respond accordingly. The server that is defined using the Flask framework, accepts requests through four different routes: First at **/handshake** to allow devices to initiate the connection. Secondly, at **/challenge** to receive the device challenge. With the handshake complete, the server listens for any data that a client might send at the **/data** endpoint. Finally, a device can also decide to end a session by calling the **/end** endpoint with its current session ID. This will initiate the vault update on the server side.

With the risk of being redundant, it needs to be said that an endpoint that will initiate the vault update simply upon receiving a valid session ID should only be used in a proof-of-concept scenario. In a real-world application, such a request must be checked for its integrity.

The client

The **client** subdirectory contains a simple implementation of what a client could look like. It is intentionally interactive, prompting the user to initiate the handshake when ready. After completing the handshake, data can be exchanged between client and server. It is synchronously encrypted using the session key that was established during the handshake. All data exchanged during the session is being temporarily stored so it can be used to create the new vaults after the session is over. The current session can be ended by the user by typing the word **end.**

The vault

One of the most integral parts of the implementation is the secure vault itself. For this proof of concept, we have opted for unencrypted integer values for the vault keys. This obviously makes the vaults insecure at rest but has the big advantage of keeping the implementation simple and being able to see the vaults change. As this software-only implementation is far away from a realistic server and hardware IoT device setup, encrypting the vaults with a hard-coded password appears redundant with no added cryptographic value.

The vault contents are stored as files that are read at the beginning of the authentication protocol and then written at the end of a session. As they are not structured in any specific format, splitting the vault with the correct key size is essential.

According to Sha and Venkatesan (2018), the **mutation** of the vaults requires using the collected session data as a key in the HMAC calculation. While this is rather vague, a simple implementation of it is concatenating all the data that is respectively sent to or received at the **/data** endpoint to a string, converting it to binary, and using it to create the new vaults.

The encryption

Our implementation uses AES-GCM encryption both during the handshake and during the following data exchange. As it is not clearly defined how to implement symmetric encryption, we opted for this Galois/Counter mode of AES that comes with additional integrity protection. The tag that is produced additionally to the ciphertext is evaluated during each encryption step and the handshake aborts if verification fails.

Tamarin prover

The Tamarin Prover is a formal verification tool for the symbolic modeling and analysis of security protocols. It allows the user to model protocol participants (e.g., initiator and responder), define the adversary's capabilities, and specify desired security properties such as secrecy or authentication. Based on this model, Tamarin automatically attempts to prove whether the protocol satisfies these properties, even under concurrent executions and active adversarial behavior.

Tamarin supports the analysis of protocols in the presence of a Dolev–Yao adversary, who can intercept, modify, and inject messages arbitrarily.

In this project, we used Tamarin to formally verify secrecy of the shared encryption key. Specifically, we proved that an adversary cannot obtain the shared key, even when able to read all messages exchanged over the network in a classical MITM scenario. This was done by modelling the protocol steps with Tamarin's proprietary scripting language and then verifying the protocols security using the powerful "autoprove" function.

Extensive documentation is available at: https://tamarin-prover.com

The tamarin prover was installed and ran locally on a consumer grade laptop. We used the Tamarin graphical interface to conduct the proof of the protocol.

Experiments

The full code that implements the Protocol and the Tamarin Script can be found in the GitHub Repository: https://github.com/oppermax/iot-secure-vault

Verification of the protocol

As mentioned, we used Tamarin prover as a tool to formally verify the security of the protocol. Specifically, we wanted to verify the claim that in a MITM attack an attacker “can’t retrieve session key t from the messages exchanged between the server and the IoT device“(Sha and Venkatesan, 2018).

To prove a claim about a protocol in Tamarin we must formulate a so-called *Theory* in form of a script which consists out of different elements:

1. Functions used (built-in and custom)
2. Rules
3. Lemmas

Functions are operations that can be used on variables as part of a rule. From Tamarin’s built-in functions, we used *symmetric-encryption* to model the encryption and decryption of the messages that are sent after M2 in the protocol. Additionally, we defined the custom functions *solve_challenge* and *xor_keys* which both accept two input values. *solve_challenge* is used to model solving the challenges proposed by the server to the device using the vault and vice versa. *xor_keys* is used to model the computation of the shared session key *t* out of *t1* and *t2*.

These custom functions do not implement the exact cryptographic computations described in the reference paper (such as iterated XOR operations over indexed vault keys), but instead abstract their behavior symbolically, which is sufficient for reasoning about security properties while keeping the model computationally tractable.

Rules describe the state transitions of the protocol and specify how messages are sent, received, and how internal state evolves during execution. In our script, each rule models one step of the IoT authentication protocol (M1–M4 + receiving M4), capturing both the communication between device and server and the corresponding updates of session state and key material.

Note on the **modelling of the vault**: The vault was deliberately modeled in an abstract manner as a single fresh, random value shared between the device and the server at the beginning of the protocol. This abstraction is justified because the internal structure and update mechanism of the vault (e.g., its composition of multiple keys and XOR operations) do not influence the core security property under investigation - namely, resistance against a MITM attacker. For the purpose of verifying secrecy of the session key in the symbolic model, it is sufficient to treat the vault as an uncompromised shared secret, allowing us to focus on protocol logic rather than low-level cryptographic computation.

Procol steps modelled in Tamarin:

1. **M1:** Device_Hello
    - Device generates fresh session id *sid*
    - Sends M1 containing *sid* and device id *did*

- Vault is set to the fresh variable *v* in device and server

2. **M2:** Receive_Device_Hello_Response
    - Server receives M1 and generates Challenge *c1* and fresh value *r1*
    - Sends M2 containing *c1* and *r1*
3. **M3:** Device_Challenge_Response
    - Device solves the challenge using the vault to derive key *k1*
    - Generates new Challenge *c2* and fresh values *t1, r2*
    - Sends message M3 encrypted with *k1*
4. **M4:** Server_Challenge_Response
    - Server receives M3 and decrypts it using the derived key *k1*
    - Verifies that the received value *r1* matches the original challenge
    - Solves challenge *c2* using the vault to derive key *k2*
    - Computes the shared session key *t* from *t1* and *t2*
    - Sends message M4 containing *r2* and fresh value *t2* encrypted with *k2*
5. **Receive M4:** Device_Finalize
    - Device receives M4 and decrypts it using the derived key *k2*
    - Verifies that the received value *r2* matches its original challenge
    - Computes the shared session key *t* from *t1* and *t2*
    - Completes the session and stores the established session key *t*

Now that we modelled the steps of the protocol, we can use these rules to define Lemmas.

Lemmas specify the security properties that the protocol must satisfy and define the statements that Tamarin attempts to prove. In this project, the lemmas were used to verify both executability of the modeled protocol and secrecy of the derived session key.

Three lemmas were used while two of them are only for meant to assist the main proof of session key secrecy.

1. **Session executability**

    This lemma verifies that the protocol is executable as modeled. It ensures that a trace - meaning a possible sequence of rule applications - exists in which an authenticated session is successfully established. Concretely, we check that a trace exists where the rules *Device_Hello, Server_Challenge_Response* and *Device_Finalize* are executed with the same values for did, sid, and t. In Tamarin, execution of a rule is referenced using action facts defined in the form `--[ ActionFact(variable1, variable2) ]->`.

    Proving this lemma confirms that the protocol model can complete a full execution.

2. **Parsing of r2**

    This lemma is a sources/typing hint that constrains where the value *r2* can come from when the server parses message M3. It states that whenever the action fact *M3Parsed(r2)*

occurs (i.e. the server has accepted an M3 containing that *r2*), then either a device previously sent an M3 with the same *r2* (recorded by *M3Sent(r2)*) or the attacker already knew *r2* (*K(r2)*), and in both cases the origin must be strictly earlier in the trace.

This lemma keeps the proof search for the final lemma finite by preventing unexplained fresh values from being injected into the server's M3 parsing step.

3. **Session key secrecy**

   This lemma states that the established session key remains secret: in any trace where the server and device complete a session using the same identifiers and key, the adversary cannot learn that key. Concretely, it rules out executions in which a matching pair of session creations occurs, and the intruder knowledge contains the corresponding session key.

By proving that no such trace exists, the lemma confirms that the protocol's key agreement produces a session key that is not learned or derivable by the attacker during a successful session establishment. This demonstrates resistance against a classical MITM attacker under the symbolic Dolev Yao model.

The powerful Tamarin autoprove function was used to prove these three lemmas in the order presented above. The autoprove procedure automatically explores the protocol's state space and attempts to either construct a proof of the lemma or produce a counterexample trace. First, the lemmas on session executability and parsing of *r2* were verified, as they establish that the protocol model is well formed and that the proof search remains finite and structured. These supporting lemmas simplify and guide the reasoning process, thereby enabling the successful automatic proof of the final and central lemma on session key secrecy.

Replication of the protocol verification

Using the file *tamarin/iot-auth.spthy* included in our git repository, it is possible to replicate the prove in Tamarin with the following steps:

1. Install Tamarin prover following the instructions found here: https://tamarin-prover.com/manual/master/book/002_installation.html
2. Using the Terminal navigate to the location of the .spthy file and enter the command `tamarin-prover interactive iot-auth.spthy`
3. The graphic interface is now available locally under: http://localhost:3001
4. Select Theory *IoT_Auth_Protocol* and then to prove each lemma click below it *sorry* > *a. autoprove*

Results and Discussion

Verification of the Protocol using Tamarin

Using Tamarins autoprove function we were able to prove the secrecy of the session key and therefore the resilience of the protocol against a MITM Attack.

The actual autoprove results are attached in the **Appendix** Section.

Session executability

The autoprove procedure successfully proved the lemma *Session_executability* by constructing a valid trace. As shown in the resulting graph (Figure 1 - Session executability result), Tamarin was able to generate a complete authentication session. This confirms that the protocol model is correctly specified and executable.

Parsing of r2

The autoprove procedure successfully established the lemma *M3_r2_sources* using induction over all possible traces (Figure 2 - Parsing of r2 result). Tamarin analyzed both the empty trace and non-empty trace cases and systematically eliminated all branches that would allow *r2* to appear in *M3Parsed(r2)* without a valid prior origin.

In every explored case, the proof showed that *r2* must either stem from a previous *M3Sent(r2)* event generated by the device or already be part of the attacker's knowledge, and that this origin must occur strictly earlier in the trace. All alternative derivations led to contradictions or cyclic dependencies and were therefore ruled out automatically.

This confirms that the model does not allow unexplained fresh values to be injected into the server's parsing step and ensures a well-structured and finite proof search for the subsequent session key secrecy lemma.

Session key secrecy

Finally, the autoprove procedure successfully proved the lemma Session_key_secrecy by showing that no trace exists in which both parties establish a session and the attacker learns the corresponding session key. During the proof, Tamarin explored all possible derivations of the key and demonstrated that learning *t* would require knowledge of the underlying vault value, which is never revealed. All attempted attack paths were reduced to contradictions, confirming that the session key remains secret under the symbolic adversary model.

Tamarin proved to be a powerful and versatile tool for verifying protocols such as the one proposed by Sha and Venkatesan (2018). Using the tool, we were able to confirm in a formal method-based approach that the claim of the protocol being secure against MITM attacks.

It is important to note that the proof holds within the symbolic model and does not account for computational attacks or implementation flaws.

Vault Security at rest

As mentioned above, the way that we implemented the secure vaults is not secure at rest. The fundamental assumption of the secure vault approach is that they are not known to third parties. While the authors do not describe precisely implementing this, one way to adapt our implementation to be closer to production readiness would be to encrypt the vault files using a password-based approach. As there is a higher risk of exploitation on the client side, as an

attacker could potentially gain access to the IoT device, the password must be stored within a hardware-based security module and only injected into the environment. Such an implementation is out of scope for this project.

Deployment of the vaults

An obvious hurdle of this approach is how to provision a new IoT device. While the paper assumes that an initial vault is already deployed on a client device, in real-life applications, a key distribution method must be used. The deployment of keys to IoT devices depends greatly on the application of the device. In industrial scenarios, factory provision where the initial vault state is stored on the device during manufacturing, can be used. Each manufactured device must then be preregistered on the server side. Especially in smart home technology, where users set up multiple different kinds of devices in their local network, out of band registration is often utilized. With this approach, the device is registered through another channel than a direct internet connection to a server but by connecting the IoT device to another trusted device with a cable, Bluetooth or a local network. Finally, over-the-air registration is possible through asymmetric cryptographic operations but is a rather resource intensive approach which defeats the secure vault approach as it is designed to be efficient and deployable to limited, possibly battery powered devices.

Two generals problem (vault synchronization)

While the authentication protocol in the reference paper and our proof using Tamarin both assume a perfect network, in a real-world scenario, this assumption surely does not hold. There are at least two obvious points of time where a network interruption would cause the server's and the client's vault to go out-of-sync:

Firstly, as the session data is used to generate the new vaults after the session ends, any failed request during the data exchange will cause the vaults to become out of sync after the session ends. This is because the entirety of the session data is used in the calculation of the new vaults. A simple way to mitigate this could be by using the session key t as a key to generate the new vaults. Because the session key is already the result of several cryptographic operations during the handshake, this arguably notable gain in resilience would hardly be a reduction of security.

Secondly, if the connection is interrupted after the client initiates the session end but before the server receives it, the client would update the vault, but the server would not. While there is no simple way to mitigate this, the way our current implementation only updates the vault states when explicitly requested is already a resilience gain.

References

Roy, K.S., Deb, S., Kalita, H.K., 2024. A novel hybrid authentication protocol utilizing lattice-based cryptography for IoT devices in fog networks. Digit. Commun. Netw. 10, 989–1000. https://doi.org/10.1016/j.dcan.2022.12.003

Shah, T., Venkatesan, S., 2018. Authentication of IoT Device and IoT Server Using Secure Vaults, in: 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/ 12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE). Presented at the 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/ 12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE), IEEE, New York, NY, USA, pp. 819–824. https://doi.org/10.1109/TrustCom/BigDataSE.2018.00117

Appendix

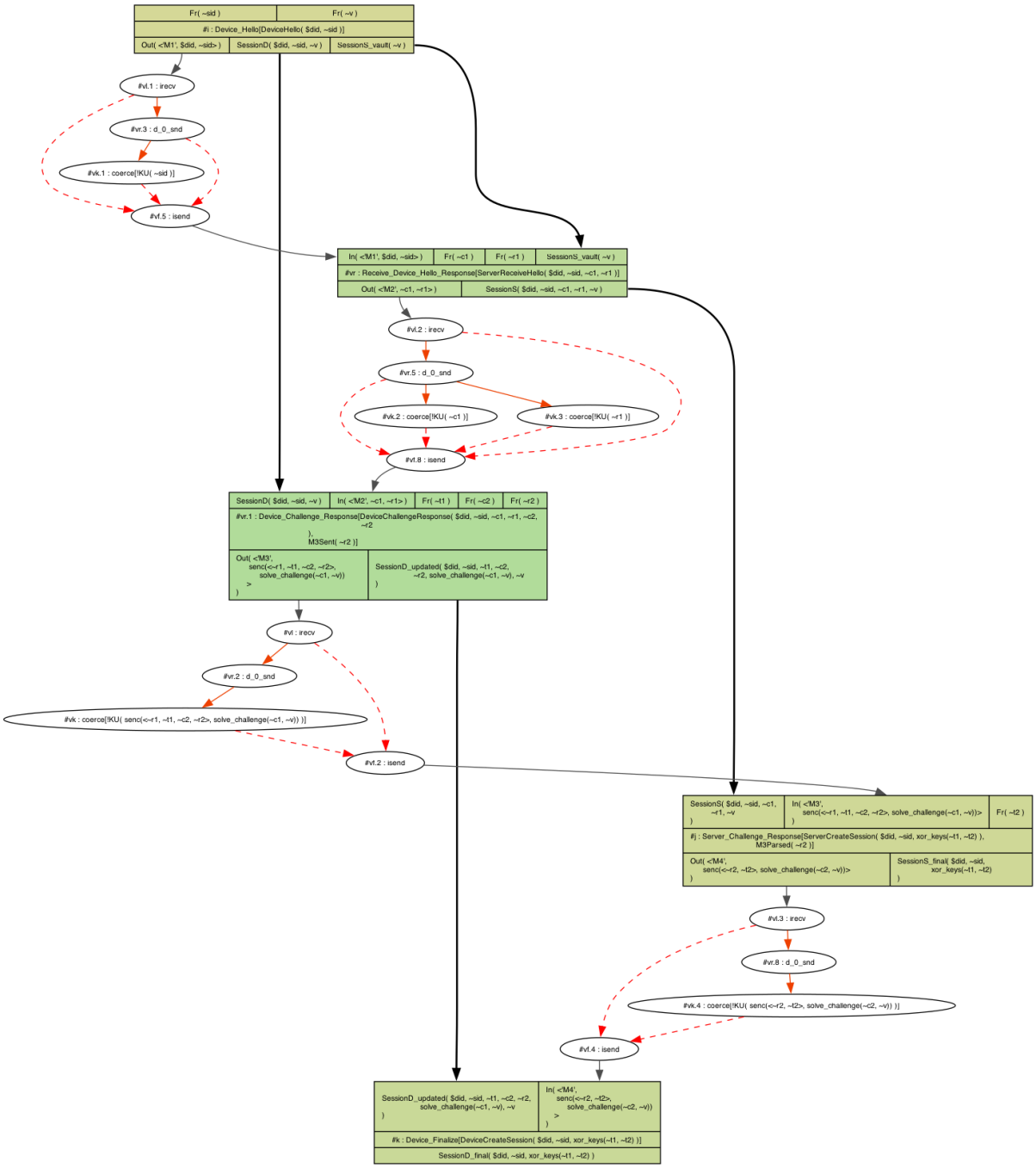

Figure 1 - Session executability result

```
lemma M3_r2_sources [sources]:
  all-traces
  "∀ r2 #i.
         (M3Parsed( r2 ) @ #i) ⇒
         ((∃ #j. (M3Sent( r2 ) @ #j) ∧ (#j < #i)) ∨
          (∃ #j. (K( r2 ) @ #j) ∧ (#j < #i)))"
induction
  case empty_trace
  by contradiction /* from formulas */
next
  case non_empty_trace
  simplify
  solve( (last(#i)) ∥
         (∃ #j. (M3Sent( r2 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #i)) ∥
         (∃ #j. (K( r2 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #i)) )
    case case_1
    solve( SessionS( $did, sid, c1, r1, v ) ▶₀ #i )
      case Receive_Device_Hello_Response
      solve( !KU( senc(<~r1, t1, c2, r2>, solve_challenge(~c1, ~v))
             ) @ #vk.2 )
        case Device_Challenge_Response
        by contradiction /* from formulas */
      next
        case Server_Challenge_Response
        solve( (∃ #j.
                  (M3Sent( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) ∥
                (∃ #j. (K( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) )
          case case_1
          by contradiction /* impossible chain */
        next
          case case_2
          by contradiction /* cyclic */
        qed
      next
        case c_senc
        solve( !KU( solve_challenge(~c1, ~v) ) @ #vk.9 )
          case Server_Challenge_Response
          solve( (∃ #j.
                    (M3Sent( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) ∥
                  (∃ #j. (K( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) )
            case case_1
            by contradiction /* impossible chain */
          next
            case case_2
            by contradiction /* cyclic */
          qed
        next
          case c_solve_challenge
          solve( !KU( ~v ) @ #vk.17 )
            case Server_Challenge_Response
            solve( (∃ #j.
                      (M3Sent( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) ∥
                    (∃ #j. (K( r2.1 ) @ #j) ∧ (¬(last(#j))) ∧ (#j < #vr.2)) )
              case case_1
              by solve( (#vr.5, 0) ~~> (#vk.2, 0) )
            next
              case case_2
              by contradiction /* cyclic */
            qed
          qed
        qed
      qed
    qed
  next
    case case_2
    by contradiction /* from formulas */
  next
    case case_3
    by contradiction /* from formulas */
  qed
qed
```

Figure 2 - Parsing of r2 result

```
lemma Session_key_secrecy:
  all-traces
  "¬(∃ did sid t #i #j #k.
          ((ServerCreateSession( did, sid, t ) @ #i) ∧
           (DeviceCreateSession( did, sid, t ) @ #j)) ∧
          (K( t ) @ #k))"
simplify
solve( SessionS( $did, sid, c1, r1, v ) ▶₀ #i )
  case Receive_Device_Hello_Response
  solve( SessionD_updated( $did, sid, t1, c2.1, r2.1, k1, v.1
         ) ▶₀ #j )
    case Device_Challenge_Response
    solve( !KU( xor_keys(~t1, ~t2) ) @ #vk.6 )
      case c_xor_keys
      solve( !KU( ~t1 ) @ #vk.17 )
        case Device_Challenge_Response
        solve( !KU( ~t2 ) @ #vk.19 )
          case Server_Challenge_Response
          by solve( !KU( ~v.1 ) @ #vk.21 )
        qed
      qed
    qed
  qed
qed
```

Figure 3 - Session key secrecy result